

Studentų duomenų valdymo programa

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 ManoVektorius< T > Class Template Reference	7
4.1.1 Member Typedef Documentation	8
4.1.1.1 const_iterator	8
4.1.1.2 iterator	8
4.1.1.3 value_type	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 ManoVektorius() [1/4]	8
4.1.2.2 ManoVektorius() [2/4]	8
4.1.2.3 ManoVektorius() [3/4]	8
4.1.2.4 ManoVektorius() [4/4]	8
4.1.2.5 ~ManoVektorius()	8
4.1.3 Member Function Documentation	9
4.1.3.1 at()	9
4.1.3.2 back() [1/2]	9
4.1.3.3 back() [2/2]	9
4.1.3.4 begin() [1/2]	9
4.1.3.5 begin() [2/2]	9
4.1.3.6 capacity()	9
4.1.3.7 clear()	9
4.1.3.8 empty()	9
4.1.3.9 end() [1/2]	10
4.1.3.10 end() [2/2]	10
4.1.3.11 erase()	10
4.1.3.12 getReallocationCount()	10
4.1.3.13 operator=() [1/2]	10
4.1.3.14 operator=() [2/2]	10
4.1.3.15 operator[]() [1/2]	10
4.1.3.16 operator[]() [2/2]	10
4.1.3.17 pop_back()	11
4.1.3.18 push_back() [1/2]	11
4.1.3.19 push_back() [2/2]	11
4.1.3.20 reallocate()	11
4.1.3.21 size()	11

4.2 Studentas Class Reference	11
4.2.1 Constructor & Destructor Documentation	13
4.2.1.1 Studentas() [1/4]	13
4.2.1.2 Studentas() [2/4]	13
4.2.1.3 ~Studentas()	13
4.2.1.4 Studentas() [3/4]	13
4.2.1.5 Studentas() [4/4]	13
4.2.2 Member Function Documentation	13
4.2.2.1 getEgzaminas()	13
4.2.2.2 getHw()	13
4.2.2.3 getPavarde()	13
4.2.2.4 getPazM()	14
4.2.2.5 getPazVid()	14
4.2.2.6 getVardas()	14
4.2.2.7 istrintiPaskutiniHw()	14
4.2.2.8 iv1()	14
4.2.2.9 iv2()	14
4.2.2.10 iv3()	14
4.2.2.11 iv4()	14
4.2.2.12 mediana()	15
4.2.2.13 operator=() [1/2]	15
4.2.2.14 operator=() [2/2]	15
4.2.2.15 pazymioPridejimas()	15
4.2.2.16 pazymys_mediana()	15
4.2.2.17 pazymys_vidurkis()	15
4.2.2.18 setEgzaminas()	15
4.2.2.19 setHw()	15
4.2.2.20 setPavarde()	15
4.2.2.21 setPazM()	16
4.2.2.22 setPazVid()	16
4.2.2.23 setVardas()	16
4.2.2.24 test()	16
4.2.2.25 vidurkis()	16
4.2.3 Friends And Related Symbol Documentation	16
4.2.3.1 operator<<	16
4.2.3.2 operator>>	16
4.3 Zmogus Class Reference	17
4.3.1 Constructor & Destructor Documentation	17
4.3.1.1 Zmogus() [1/3]	17
4.3.1.2 ~Zmogus()	17
4.3.1.3 Zmogus() [2/3]	17
4.3.1.4 Zmogus() [3/3]	18

4.3.2 Member Function Documentation	18
4.3.2.1 getPavarde()	18
4.3.2.2 getPazM()	18
4.3.2.3 getPazVid()	18
4.3.2.4 getVardas()	18
4.3.2.5 operator=()	18
4.3.2.6 setPavarde()	18
4.3.2.7 setVardas()	19
4.3.3 Member Data Documentation	19
4.3.3.1 pavarde	19
4.3.3.2 vardas	19
5 File Documentation	21
5.1 vektoriai/code.h File Reference	21
5.1.1 Function Documentation	21
5.1.1.1 apdorojimo_laikas()	21
5.1.1.2 generavimas()	21
5.1.1.3 raide()	21
5.2 code.h	22
5.3 vektoriai/functions.cpp File Reference	22
5.3.1 Function Documentation	22
5.3.1.1 apdorojimo_laikas()	22
5.3.1.2 generavimas()	22
5.3.1.3 raide()	22
5.4 vektoriai/my.h File Reference	23
5.5 my.h	23
5.6 vektoriai/students.cpp File Reference	24
5.6.1 Function Documentation	24
5.6.1.1 operator<<()	24
5.6.1.2 operator>>()	24
5.7 vektoriai/students.h File Reference	24
5.8 students.h	25
5.9 vektoriai/vector.h File Reference	26
5.10 vector.h	26
5.11 vektoriai/vektoriuMain.cpp File Reference	29
5.11.1 Function Documentation	30
5.11.1.1 main()	30
5.12 vektoriai/Zmogus.h File Reference	30
5.12.1 Function Documentation	30
5.12.1.1 operator<<()	30
5.13 Zmogus.h	30
Index	33

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

ManoVektorius< T >	7
Zmogus	17
Studentas	11

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ManoVektorius< T >	7
Studentas	11
Zmogus	17

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

vektoriai/ code.h	21
vektoriai/ functions.cpp	22
vektoriai/ my.h	23
vektoriai/ students.cpp	24
vektoriai/ students.h	24
vektoriai/ vector.h	26
vektoriai/ vektoriuMain.cpp	29
vektoriai/ Zmogus.h	30

Chapter 4

Class Documentation

4.1 ManoVektorius< T > Class Template Reference

```
#include <vector.h>
```

Public Types

- using `value_type` = T
- using `iterator` = T*
- using `const_iterator` = const T*

Public Member Functions

- `ManoVektorius` ()
- `ManoVektorius` (std::initializer_list< T > list)
- `ManoVektorius` (const `ManoVektorius` &other)
- `ManoVektorius` & `operator=` (const `ManoVektorius` &other)
- `ManoVektorius` (`ManoVektorius` &&other) noexcept
- `ManoVektorius` & `operator=` (`ManoVektorius` &&other) noexcept
- `~ManoVektorius` ()
- `size_t` `getReallocationCount` () const
- void `push_back` (const T &value)
- void `push_back` (T &&value)
- void `pop_back` ()
- void `reallocate` (size_t new_capacity)
- void `clear` ()
- void `erase` (size_t index)
- T & `operator[]` (size_t index)
- const T & `operator[]` (size_t index) const
- T & `at` (size_t index)
- `size_t` `size` () const noexcept
- `size_t` `capacity` () const noexcept
- bool `empty` () const noexcept
- const T * `end` () const
- const T * `begin` () const
- T * `begin` ()
- T * `end` ()
- T & `back` ()
- const T & `back` () const

4.1.1 Member Typedef Documentation

4.1.1.1 `const_iterator`

```
template<typename T>
using ManoVektorius< T >::const_iterator = const T*
```

4.1.1.2 `iterator`

```
template<typename T>
using ManoVektorius< T >::iterator = T*
```

4.1.1.3 `value_type`

```
template<typename T>
using ManoVektorius< T >::value_type = T
```

4.1.2 Constructor & Destructor Documentation

4.1.2.1 `ManoVektorius()` [1/4]

```
template<typename T>
ManoVektorius< T >::ManoVektorius ()
```

4.1.2.2 `ManoVektorius()` [2/4]

```
template<typename T>
ManoVektorius< T >::ManoVektorius (
    std::initializer_list< T > list) [inline]
```

4.1.2.3 `ManoVektorius()` [3/4]

```
template<typename T>
ManoVektorius< T >::ManoVektorius (
    const ManoVektorius< T > & other)
```

4.1.2.4 `ManoVektorius()` [4/4]

```
template<typename T>
ManoVektorius< T >::ManoVektorius (
    ManoVektorius< T > && other) [noexcept]
```

4.1.2.5 `~ManoVektorius()`

```
template<typename T>
ManoVektorius< T >::~~ManoVektorius ()
```

4.1.3 Member Function Documentation

4.1.3.1 at()

```
template<typename T>
T & ManoVektorius< T >::at (
    size_t index)
```

4.1.3.2 back() [1/2]

```
template<typename T>
T & ManoVektorius< T >::back ()
```

4.1.3.3 back() [2/2]

```
template<typename T>
const T & ManoVektorius< T >::back () const
```

4.1.3.4 begin() [1/2]

```
template<typename T>
T * ManoVektorius< T >::begin ()
```

4.1.3.5 begin() [2/2]

```
template<typename T>
const T * ManoVektorius< T >::begin () const
```

4.1.3.6 capacity()

```
template<typename T>
size_t ManoVektorius< T >::capacity () const [noexcept]
```

4.1.3.7 clear()

```
template<typename T>
void ManoVektorius< T >::clear ()
```

4.1.3.8 empty()

```
template<typename T>
bool ManoVektorius< T >::empty () const [noexcept]
```

4.1.3.9 end() [1/2]

```
template<typename T>
T * ManoVektorius< T >::end ()
```

4.1.3.10 end() [2/2]

```
template<typename T>
const T * ManoVektorius< T >::end () const
```

4.1.3.11 erase()

```
template<typename T>
void ManoVektorius< T >::erase (
    size_t index)
```

4.1.3.12 getReallocationCount()

```
template<typename T>
size_t ManoVektorius< T >::getReallocationCount () const [inline]
```

4.1.3.13 operator=() [1/2]

```
template<typename T>
ManoVektorius< T > & ManoVektorius< T >::operator= (
    const ManoVektorius< T > & other)
```

4.1.3.14 operator=() [2/2]

```
template<typename T>
ManoVektorius< T > & ManoVektorius< T >::operator= (
    ManoVektorius< T > && other) [noexcept]
```

4.1.3.15 operator[]() [1/2]

```
template<typename T>
T & ManoVektorius< T >::operator[] (
    size_t index)
```

4.1.3.16 operator[]() [2/2]

```
template<typename T>
const T & ManoVektorius< T >::operator[] (
    size_t index) const
```


4.1.3.17 pop_back()

```
template<typename T>
void ManoVektorius< T >::pop_back ()
```

4.1.3.18 push_back() [1/2]

```
template<typename T>
void ManoVektorius< T >::push_back (
    const T & value)
```

4.1.3.19 push_back() [2/2]

```
template<typename T>
void ManoVektorius< T >::push_back (
    T && value)
```

4.1.3.20 reallocate()

```
template<typename T>
void ManoVektorius< T >::reallocate (
    size_t new_capacity)
```

4.1.3.21 size()

```
template<typename T>
size_t ManoVektorius< T >::size () const [noexcept]
```

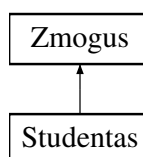
The documentation for this class was generated from the following file:

- vektoriai/[vector.h](#)

4.2 Studentas Class Reference

```
#include <students.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (const string &[vardas](#), const string &[pavarde](#), const [ManoVektorius](#)< int > &hw, int egzaminas)
- [~Studentas](#) ()
- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other) noexcept
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
- string [getVardas](#) () const override
- string [getPavarde](#) () const override
- double [getPazVid](#) () const override
- double [getPazM](#) () const override
- int [getEgzaminas](#) () const
- [ManoVektorius](#)< int > [getHw](#) () const
- void [setVardas](#) (const string &v) override
- void [setPavarde](#) (const string &p) override
- void [setPazVid](#) (const double &vid)
- void [setPazM](#) (const double &med)
- void [setEgzaminas](#) (const int &egz)
- void [setHw](#) (const [ManoVektorius](#)< int > &nd)
- void [pazymioPridejimas](#) (int nd)
- void [istrintiPaskutiniHw](#) ()
- void [test](#) ()
- double [vidurkis](#) () const
- double [mediana](#) () const
- void [pazymys_mediana](#) ()
- void [pazymys_vidurkis](#) ()
- void [iv1](#) ()
- void [iv2](#) (mt19937 &gen)
- void [iv3](#) ([ManoVektorius](#)< string > &vardai, [ManoVektorius](#)< string > &pavardes, mt19937 &gen)
- double [iv4](#) ([ManoVektorius](#)< unique_ptr< [Zmogus](#) > > &grupe, ofstream &laiko_failas)

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) (const string &[vardas](#), const string &[pavarde](#))
- virtual [~Zmogus](#) ()=0
- [Zmogus](#) (const [Zmogus](#) &other)=default
- [Zmogus](#) ([Zmogus](#) &&other) noexcept
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other) noexcept

Friends

- ostream & [operator<<](#) (ostream &os, const [Zmogus](#) &zm)
- istream & [operator>>](#) (istream &is, [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [vardas](#)
- string [pavarde](#)

4.2.1 Constructor & Destructor Documentation

4.2.1.1 Studentas() [1/4]

```
Studentas::Studentas () [inline]
```

4.2.1.2 Studentas() [2/4]

```
Studentas::Studentas (  
    const string & vardas,  
    const string & pavarde,  
    const ManoVektorius< int > & hw,  
    int egzaminas) [inline]
```

4.2.1.3 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

4.2.1.4 Studentas() [3/4]

```
Studentas::Studentas (  
    const Studentas & other) [inline]
```

4.2.1.5 Studentas() [4/4]

```
Studentas::Studentas (  
    Studentas && other) [inline], [noexcept]
```

4.2.2 Member Function Documentation

4.2.2.1 getEgzaminas()

```
int Studentas::getEgzaminas () const [inline]
```

4.2.2.2 getHw()

```
ManoVektorius< int > Studentas::getHw () const [inline]
```

4.2.2.3 getPavarde()

```
string Studentas::getPavarde () const [override], [virtual]
```

Reimplemented from [Zmogus](#).

4.2.2.4 getPazM()

```
double Studentas::getPazM () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.2.2.5 getPazVid()

```
double Studentas::getPazVid () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.2.2.6 getVardas()

```
string Studentas::getVardas () const [override], [virtual]
```

Reimplemented from [Zmogus](#).

4.2.2.7 istrintiPaskutiniHw()

```
void Studentas::istrintiPaskutiniHw () [inline]
```

4.2.2.8 iv1()

```
void Studentas::iv1 ()
```

4.2.2.9 iv2()

```
void Studentas::iv2 (  
    mt19937 & gen)
```

4.2.2.10 iv3()

```
void Studentas::iv3 (  
    ManoVektorius< string > & vardai,  
    ManoVektorius< string > & pavardes,  
    mt19937 & gen)
```

4.2.2.11 iv4()

```
double Studentas::iv4 (  
    ManoVektorius< unique_ptr< Zmogus > > & grupe,  
    ofstream & laiko_failas)
```

4.2.2.12 mediana()

```
double Studentas::mediana () const
```

4.2.2.13 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other) [inline]
```

4.2.2.14 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [inline], [noexcept]
```

4.2.2.15 pazymioPridejimas()

```
void Studentas::pazymioPridejimas (
    int nd) [inline]
```

4.2.2.16 pazymys_mediana()

```
void Studentas::pazymys_mediana ()
```

4.2.2.17 pazymys_vidurkis()

```
void Studentas::pazymys_vidurkis ()
```

4.2.2.18 setEgzaminas()

```
void Studentas::setEgzaminas (
    const int & egz) [inline]
```

4.2.2.19 setHw()

```
void Studentas::setHw (
    const ManoVektorius< int > & nd) [inline]
```

4.2.2.20 setPavarde()

```
void Studentas::setPavarde (
    const string & p) [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.2.2.21 setPazM()

```
void Studentas::setPazM (
    const double & med) [inline]
```

4.2.2.22 setPazVid()

```
void Studentas::setPazVid (
    const double & vid) [inline]
```

4.2.2.23 setVardas()

```
void Studentas::setVardas (
    const string & v) [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.2.2.24 test()

```
void Studentas::test ()
```

4.2.2.25 vidurkis()

```
double Studentas::vidurkis () const
```

4.2.3 Friends And Related Symbol Documentation

4.2.3.1 operator<<

```
ostream & operator<< (
    ostream & os,
    const Zmogus & zm) [friend]
```

4.2.3.2 operator>>

```
istream & operator>> (
    istream & is,
    Studentas & s) [friend]
```

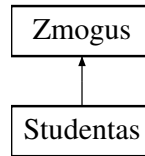
The documentation for this class was generated from the following files:

- [vektoriai/students.h](#)
- [vektoriai/students.cpp](#)

4.3 Zmogus Class Reference

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- `Zmogus` (const string &`vardas`, const string &`pavarde`)
- virtual `~Zmogus` ()=0
- virtual string `getVardas` () const
- virtual string `getPavarde` () const
- virtual double `getPazVid` () const =0
- virtual double `getPazM` () const =0
- virtual void `setVardas` (const string &`v`)=0
- virtual void `setPavarde` (const string &`p`)=0
- `Zmogus` (const `Zmogus` &`other`)=default
- `Zmogus` (`Zmogus` &&`other`) noexcept
- `Zmogus` & `operator=` (`Zmogus` &&`other`) noexcept

Protected Attributes

- string `vardas`
- string `pavarde`

4.3.1 Constructor & Destructor Documentation

4.3.1.1 `Zmogus()` [1/3]

```
Zmogus::Zmogus (
    const string & vardas,
    const string & pavarde) [inline]
```

4.3.1.2 `~Zmogus()`

```
Zmogus::~~Zmogus () [pure virtual]
```

4.3.1.3 `Zmogus()` [2/3]

```
Zmogus::Zmogus (
    const Zmogus & other) [default]
```

4.3.1.4 Zmogus() [3/3]

```
Zmogus::Zmogus (  
    Zmogus && other) [inline], [noexcept]
```

4.3.2 Member Function Documentation

4.3.2.1 getPavarde()

```
virtual string Zmogus::getPavarde () const [inline], [virtual]
```

Reimplemented in [Studentas](#).

4.3.2.2 getPazM()

```
virtual double Zmogus::getPazM () const [pure virtual]
```

Implemented in [Studentas](#).

4.3.2.3 getPazVid()

```
virtual double Zmogus::getPazVid () const [pure virtual]
```

Implemented in [Studentas](#).

4.3.2.4 getVardas()

```
virtual string Zmogus::getVardas () const [inline], [virtual]
```

Reimplemented in [Studentas](#).

4.3.2.5 operator=()

```
Zmogus & Zmogus::operator= (  
    Zmogus && other) [inline], [noexcept]
```

4.3.2.6 setPavarde()

```
virtual void Zmogus::setPavarde (  
    const string & p) [pure virtual]
```

Implemented in [Studentas](#).

4.3.2.7 setVardas()

```
virtual void Zmogus::setVardas (
    const string & v) [pure virtual]
```

Implemented in [Studentas](#).

4.3.3 Member Data Documentation

4.3.3.1 pavarde

```
string Zmogus::pavarde [protected]
```

4.3.3.2 vardas

```
string Zmogus::vardas [protected]
```

The documentation for this class was generated from the following files:

- [vektoriai/Zmogus.h](#)
- [vektoriai/students.cpp](#)

Chapter 5

File Documentation

5.1 vektoriai/code.h File Reference

```
#include "my.h"
```

Functions

- string [raide](#) (string vardai)
- void [generavimas](#) (string failas, int dydis)
- double [apdorojimo_laikas](#) (high_resolution_clock::time_point start, high_resolution_clock::time_point end)

5.1.1 Function Documentation

5.1.1.1 apdorojimo_laikas()

```
double apdorojimo_laikas (  
    high_resolution_clock::time_point start,  
    high_resolution_clock::time_point end)
```

5.1.1.2 generavimas()

```
void generavimas (  
    string failas,  
    int dydis)
```

5.1.1.3 raide()

```
string raide (  
    string vardai)
```

5.2 code.h

[Go to the documentation of this file.](#)

```
00001
00002 #ifndef CODE_H
00003 #define CODE_H
00004 #include "my.h"
00005
00006 string raide(string vardai);
00007 void generavimas(string failas, int dydis);
00008 double apdorojimo_laikas(high_resolution_clock::time_point start, high_resolution_clock::time_point
    end);
00009 //void generavimo_laikas(string& filename, ofstream& laiko_failas);
00010 //void RAM(ofstream& report);
00011 //void CPU(ofstream& report);
00012 //void Disk(ofstream& report);
00013
00014
00015 #endif
```

5.3 vektoriai/functions.cpp File Reference

```
#include "my.h"
#include "code.h"
#include "students.h"
#include "Zmogus.h"
#include "vector.h"
```

Functions

- string [raide](#) (string vardai)
- void [generavimas](#) (string failas, int dydis)
- double [apdorojimo_laikas](#) (high_resolution_clock::time_point start, high_resolution_clock::time_point end)

5.3.1 Function Documentation

5.3.1.1 apdorojimo_laikas()

```
double apdorojimo_laikas (
    high_resolution_clock::time_point start,
    high_resolution_clock::time_point end)
```

5.3.1.2 generavimas()

```
void generavimas (
    string failas,
    int dydis)
```

5.3.1.3 raide()

```
string raide (
    string vardai)
```

5.4 vektoriai/my.h File Reference

```
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>
#include <string>
#include <algorithm>
#include <ctime>
#include <chrono>
#include <numeric>
#include <random>
#include <sstream>
#include <windows.h>
#include <cassert>
#include <memory>
#include <utility>
```

5.5 my.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MY_H
00002 #define MY_H
00003 #include <iostream>
00004 #include <fstream>
00005 #include <vector>
00006 #include <iomanip>
00007 #include <string>
00008 #include <algorithm>
00009 #include <ctime>
00010 #include <chrono>
00011 #include <numeric>
00012 #include <random>
00013 #include <sstream>
00014 #include <windows.h>
00015 #include <cassert>
00016 #include <memory>
00017 #include <utility>
00018
00019 using std::cout;
00020 using std::cin;
00021 using std::string;
00022 using std::vector;
00023 using std::endl;
00024 using std::fixed;
00025 using std::setprecision;
00026 using std::left;
00027 using std::setw;
00028 using std::sort;
00029 using std::ifstream;
00030 using std::ofstream;
00031 using std::transform;
00032 using std::exception;
00033 using std::cerr;
00034 using std::invalid_argument;
00035 using std::out_of_range;
00036 using std::to_string;
00037 using namespace std::chrono;
00038 using std::accumulate;
00039 using std::random_device;
00040 using std::mt19937;
00041 using std::uniform_int_distribution;
00042 using std::uniform_real_distribution;
00043 using std::runtime_error;
00044 using std::stringstream;
00045 using std::ios;
00046 using std::copy_if;
00047 using std::back_inserter;
00048 using std::remove_if;
00049 using std::istreamstream;
```

```
00050 using std::move;
00051 using std::ostream;
00052 using std::istream;
00053 using std::getline;
00054 using std::abs;
00055 using std::logic_error;
00056 using std::unique_ptr;
00057 using std::make_unique;
00058
00059 #endif
```

5.6 vektoriai/students.cpp File Reference

```
#include "my.h"
#include "code.h"
#include "students.h"
#include "Zmogus.h"
#include <memory>
#include "vector.h"
```

Functions

- ostream & [operator<<](#) (ostream &os, const [Zmogus](#) &zm)
- istream & [operator>>](#) (istream &is, [Studentas](#) &B)

5.6.1 Function Documentation

5.6.1.1 operator<<()

```
ostream & operator<< (
    ostream & os,
    const Zmogus & zm)
```

5.6.1.2 operator>>()

```
istream & operator>> (
    istream & is,
    Studentas & B)
```

5.7 vektoriai/students.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <random>
#include <cmath>
#include "my.h"
#include "code.h"
#include "Zmogus.h"
#include "vector.h"
```

Classes

- class [Studentas](#)

5.8 students.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENTS_H
00002 #define STUDENTS_H
00003 #include <iostream>
00004 #include <string>
00005 #include <vector>
00006 #include <algorithm>
00007 #include <random>
00008 #include <cmath>
00009 #include "my.h"
00010 #include "code.h"
00011 #include "Zmogus.h"
00012 #include "vector.h"
00013
00014     class Studentas: public Zmogus{
00015         ManoVektorius <int> hw;
00016         int egzaminas;
00017         double paz_m;
00018         double paz_vid;
00019     public:
00020         // Default constructor
00021         Studentas()
00022         : Zmogus("", ""), egzaminas(0), paz_m(0), paz_vid(0) {}
00023
00024         Studentas(const string& vardas, const string& pavarde, const ManoVektorius<int>& hw, int
egzaminas)
00025         : Zmogus(vardas, pavarde), hw(hw), egzaminas(egzaminas) {}
00026
00027         ~Studentas() {
00028             hw.clear();
00029             egzaminas = 0;
00030             paz_vid = 0.0;
00031             paz_m = 0.0;
00032             vardas.clear();
00033             pavarde.clear();
00034         }
00035         // Copy constructor and assignment operator
00036         Studentas(const Studentas& other)
00037         : Zmogus(other), egzaminas(other.egzaminas), hw(other.hw),
00038           paz_m(other.paz_m), paz_vid(other.paz_vid) {}
00039
00040         Studentas& operator=(const Studentas& other) {
00041             if (this != &other) {
00042                 vardas = other.vardas;
00043                 pavarde = other.pavarde;
00044                 hw = other.hw;
00045                 egzaminas = other.egzaminas;
00046                 paz_m = other.paz_m;
00047                 paz_vid = other.paz_vid;
00048             }
00049             return *this;
00050         }
00051         // Move constructor and assignment operator
00052         Studentas(Studentas&& other) noexcept:
00053             Zmogus(move(other)),
00054             hw(move(other.hw)), egzaminas((other.egzaminas)),
00055             paz_m((other.paz_m)), paz_vid((other.paz_vid)) {
00056             other.egzaminas = 0;
00057             other.paz_m = 0.0;
00058             other.paz_vid = 0.0;
00059         }
00060
00061         Studentas& operator=(Studentas&& other) noexcept {
00062             if (this != &other) {
00063                 Zmogus::operator = (move(other));
00064
00065                 hw = move(other.hw);
00066                 egzaminas = other.egzaminas;
00067                 paz_m = other.paz_m;
00068                 paz_vid = other.paz_vid;
00069
00070                 other.egzaminas = 0;
00071                 other.paz_m = 0.0;
00072                 other.paz_vid = 0.0;

```

```

00073         }
00074         return *this;
00075     }
00076
00077     //Input and output operations
00078     friend ostream& operator<<(ostream& os, const Zmogus& zm);
00079     friend istream& operator>>(istream& is, Studentas& s);
00080
00081     // Getters
00082     string getVardas() const override;
00083     string getPavarde() const override;
00084     double getPazVid() const override { return paz_vid; }
00085     double getPazM() const override { return paz_m; }
00086     int getEgzaminas() const { return egzaminas; }
00087     ManoVektorius<int> getHw() const { return hw; }
00088
00089     //Setters
00090     void setVardas(const string& v) override { vardas = v; };
00091     void setPavarde(const string& p) override { pavarde = p; };
00092     void setPazVid(const double& vid) { paz_vid = vid; }
00093     void setPazM(const double& med) { paz_m = med; }
00094     void setEgzaminas(const int& egz) { egzaminas = egz; }
00095     void setHw(const ManoVektorius<int>& nd) { hw = nd; }
00096
00097     void pazymioPridejimas(int nd) {
00098         hw.push_back(nd);
00099     }
00100
00101     void istrintiPaskutiniHw() {
00102         if (!hw.empty()) hw.pop_back();
00103     }
00104
00105     // Methods
00106     void test ();
00107     double vidurkis() const;
00108     double mediana() const;
00109     void pazymys_mediana();
00110     void pazymys_vidurkis();
00111     void ivl();
00112     void iv2(mt19937& gen);
00113     void iv3(ManoVektorius <string>& vardai, ManoVektorius <string>& pavardes, mt19937& gen);
00114     double iv4(ManoVektorius <unique_ptr<Zmogus>& grupe, ofstream& laiko_failas);
00115
00116     };
00117
00118 #endif

```

5.9 vektoriai/vector.h File Reference

```

#include <initializer_list>
#include <algorithm>
#include <random>
#include <cmath>
#include <iostream>
#include <string>
#include <vector>
#include <utility>

```

Classes

- class [ManoVektorius< T >](#)

5.10 vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef VECTOR_H
```



```

00002 #define VECTOR_H
00003
00004 #include <initializer_list>
00005 #include <algorithm>
00006 #include <random>
00007 #include <cmath>
00008 #include <iostream>
00009 #include <string>
00010 #include <vector>
00011 #include <utility>
00012
00013 template <typename T>
00014 class ManoVektorius {
00015     private:
00016         T* data_;
00017         size_t size_;
00018         size_t capacity_;
00019         size_t realloc_count_ = 0;
00020
00021     public:
00022         using value_type = T;
00023         using iterator = T*;
00024         using const_iterator = const T*;
00025
00026         ManoVektorius();
00027         ManoVektorius(std::initializer_list<T> list) {
00028             size_ = list.size();
00029             capacity_ = size_;
00030             data_ = new T[capacity_];
00031             std::copy(list.begin(), list.end(), data_);
00032         }
00033     };
00034
00035 ManoVektorius(const ManoVektorius& other);
00036 ManoVektorius& operator=(const ManoVektorius& other);
00037 ManoVektorius(ManoVektorius&& other) noexcept; // move constructor
00038 ManoVektorius& operator=(ManoVektorius&& other) noexcept; // move assignment
00039 ~ManoVektorius();
00040
00041
00042 size_t getReallocationCount() const {
00043     return realloc_count_;
00044 }
00045
00046 void push_back(const T& value);
00047 void push_back(T&& value);
00048 void pop_back();
00049 void reallocate(size_t new_capacity);
00050 void clear();
00051 void erase(size_t index);
00052
00053 T& operator[](size_t index);
00054 const T& operator[](size_t index) const;
00055 T& at(size_t index);
00056
00057 size_t size() const noexcept;
00058 size_t capacity() const noexcept;
00059 bool empty() const noexcept;
00060
00061 const T* end() const;
00062 const T* begin() const;
00063 T* begin();
00064 T* end();
00065 T& back();
00066 const T& back() const;
00067 };
00068
00069 template <typename T>
00070 ManoVektorius<T>::ManoVektorius() : data_(nullptr), size_(0), capacity_(0) {}
00071
00072 template <typename T>
00073 ManoVektorius<T>::ManoVektorius(const ManoVektorius<T>& other)
00074     : size_(other.size_), capacity_(other.capacity_) {
00075     data_ = new T[capacity_];
00076     for (size_t i = 0; i < size_; ++i) {
00077         data_[i] = other.data_[i]; // T must be copyable
00078     }
00079 }
00080
00081 template <typename T>
00082 ManoVektorius<T>& ManoVektorius<T>::operator=(const ManoVektorius<T>& other) {
00083     if (this == &other) return *this; // self-assignment guard
00084
00085     T* new_data = new T[other.capacity_];
00086     for (size_t i = 0; i < other.size_; ++i) {
00087         new_data[i] = other.data_[i];
00088     }

```

```

00089
00090     delete[] data_; // free old memory
00091     data_ = new_data;
00092     size_ = other.size_;
00093     capacity_ = other.capacity_;
00094
00095     return *this;
00096 }
00097
00098
00099 template <typename T>
00100 ManoVektorius<T>::ManoVektorius(ManoVektorius<T>&& other) noexcept
00101 : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
00102     other.data_ = nullptr;
00103     other.size_ = 0;
00104     other.capacity_ = 0;
00105 }
00106
00107 template <typename T>
00108 ManoVektorius<T>& ManoVektorius<T>::operator=(ManoVektorius<T>&& other) noexcept {
00109     if (this != &other) {
00110         delete[] data_;
00111         data_ = other.data_;
00112         size_ = other.size_;
00113         capacity_ = other.capacity_;
00114         other.data_ = nullptr;
00115         other.size_ = 0;
00116         other.capacity_ = 0;
00117     }
00118     return *this;
00119 }
00120
00121 template <typename T>
00122 ManoVektorius<T>::~ManoVektorius() {
00123     delete[] data_;
00124 }
00125 //-----
00126 template <typename T>
00127 void ManoVektorius<T>::push_back(const T& value) {
00128     // cout<<"push_back be move\n";
00129     if (size_ == capacity_) {
00130         reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00131     }
00132     data_[size_++] = value;
00133     // cout<<"push_back be move completed\n";
00134 }
00135
00136 template <typename T>
00137 void ManoVektorius<T>::push_back(T&& value) {
00138     // cout<<"push_back su move\n";
00139     if (size_ == capacity_) {
00140         reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00141     }
00142     data_[size_++] = std::move(value);
00143     // cout<<"push_back su move completed\n";
00144 }
00145 //-----
00146 template <typename T>
00147 void ManoVektorius<T>::pop_back() {
00148     if (size_ > 0) {
00149         --size_;
00150     }
00151 }
00152
00153 template <typename T>
00154 void ManoVektorius<T>::reallocate(size_t new_capacity) {
00155     T* new_data = new T[new_capacity];
00156     for (size_t i = 0; i < size_; ++i)
00157         new_data[i] = std::move(data_[i]);
00158     delete[] data_;
00159     data_ = new_data;
00160     capacity_ = new_capacity;
00161     ++realloc_count_;
00162 }
00163
00164 template <typename T>
00165 void ManoVektorius<T>::clear() {
00166     size_ = 0;
00167 }
00168
00169 template <typename T>
00170 void ManoVektorius<T>::erase(std::size_t index) {
00171     if (index >= size_) throw std::out_of_range("Index out of bounds");
00172     for (std::size_t i = index; i < size_ - 1; ++i) {
00173         data_[i] = std::move(data_[i + 1]);
00174     }
00175     --size_;

```

```

00176 }
00177 //-----
00178 template <typename T>
00179 T& ManoVektorius<T>::operator[](size_t index) {
00180     return data_[index];
00181 }
00182
00183 template <typename T>
00184 const T& ManoVektorius<T>::operator[](size_t index) const {
00185     return data_[index];
00186 }
00187
00188 template <typename T>
00189 T& ManoVektorius<T>::at(size_t index) {
00190     if (index >= size_) throw std::out_of_range("Index out of range");
00191     return data_[index];
00192 }
00193 //-----
00194 template <typename T>
00195 size_t ManoVektorius<T>::size() const noexcept {
00196     return size_;
00197 }
00198
00199 template <typename T>
00200 size_t ManoVektorius<T>::capacity() const noexcept {
00201     return capacity_;
00202 }
00203
00204 template <typename T>
00205 bool ManoVektorius<T>::empty() const noexcept {
00206     return size_ == 0;
00207 }
00208 //-----
00209 template <typename T>
00210 T& ManoVektorius<T>::back() {
00211     return data_[size_ - 1];
00212 }
00213
00214 template <typename T>
00215 const T& ManoVektorius<T>::back() const {
00216     return data_[size_ - 1];
00217 }
00218
00219 template <typename T>
00220 T* ManoVektorius<T>::begin() {
00221     return data_;
00222 }
00223
00224 template <typename T>
00225 T* ManoVektorius<T>::end() {
00226     return data_ + size_;
00227 }
00228
00229 template <typename T>
00230 const T* ManoVektorius<T>::begin() const {
00231     return data_;
00232 }
00233
00234 template <typename T>
00235 const T* ManoVektorius<T>::end() const {
00236     return data_ + size_;
00237 }
00238
00239 #endif

```

5.11 vektoriai/vektoriuMain.cpp File Reference

```

#include "my.h"
#include "code.h"
#include "students.h"
#include "Zmogus.h"
#include "vector.h"
#include <memory>

```

Functions

- int [main](#) ()

5.11.1 Function Documentation

5.11.1.1 main()

```
int main ()
```

5.12 vektoriai/Zmogus.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <random>
#include <cmath>
#include "my.h"
#include "code.h"
```

Classes

- class [Zmogus](#)

Functions

- ostream & [operator<<](#) (ostream &os, const [Zmogus](#) &zm)

5.12.1 Function Documentation

5.12.1.1 operator<<()

```
ostream & operator<< (
    ostream & os,
    const Zmogus & zm)
```

5.13 Zmogus.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003 #include <iostream>
00004 #include <string>
00005 #include <vector>
00006 #include <algorithm>
00007 #include <random>
00008 #include <cmath>
00009 #include "my.h"
00010 #include "code.h"
00011
00012
00013 class Zmogus {
00014     protected:
00015     string vardas;
```

```
00016     string pavarde;
00017
00018     public:
00019     Zmogus(const string& vardas, const string& pavarde)
00020         : vardas(vardas), pavarde(pavarde) {}
00021
00022     virtual ~Zmogus() = 0; //pure virtual destructor
00023
00024     //Getters
00025     virtual string getVardas() const { return vardas; }
00026     virtual string getPavarde() const { return pavarde; }
00027     virtual double getPazVid() const = 0;
00028     virtual double getPazM() const = 0;
00029
00030     //Setters
00031     virtual void setVardas(const string& v) = 0;
00032     virtual void setPavarde(const string& p) = 0;
00033
00034     Zmogus(const Zmogus& other) = default;
00035
00036     Zmogus(Zmogus&& other) noexcept
00037     : vardas(move(other.vardas)), pavarde(move(other.pavarde)) {
00038     other.vardas = "";
00039     other.pavarde = "";
00040 }
00041
00042     Zmogus& operator=(Zmogus&& other) noexcept {
00043     if (this != &other) {
00044         vardas = move(other.vardas);
00045         pavarde = move(other.pavarde);
00046         other.vardas = "";
00047         other.pavarde = "";
00048     }
00049     return *this;
00050 }
00051
00052
00053 };
00054
00055 //Zmogus::~Zmogus() {} // definition of pure virtual destructor
00056 ostream& operator<<(ostream& os, const Zmogus& zm);
00057
00058
00059 #endif
```


Index

- ~ManoVektorius
 - ManoVektorius< T >, 8
- ~Studentas
 - Studentas, 13
- ~Zmogus
 - Zmogus, 17
- apdorojimo_laikas
 - code.h, 21
 - functions.cpp, 22
- at
 - ManoVektorius< T >, 9
- back
 - ManoVektorius< T >, 9
- begin
 - ManoVektorius< T >, 9
- capacity
 - ManoVektorius< T >, 9
- clear
 - ManoVektorius< T >, 9
- code.h
 - apdorojimo_laikas, 21
 - generavimas, 21
 - raide, 21
- const_iterator
 - ManoVektorius< T >, 8
- empty
 - ManoVektorius< T >, 9
- end
 - ManoVektorius< T >, 9, 10
- erase
 - ManoVektorius< T >, 10
- functions.cpp
 - apdorojimo_laikas, 22
 - generavimas, 22
 - raide, 22
- generavimas
 - code.h, 21
 - functions.cpp, 22
- getEgzaminas
 - Studentas, 13
- getHw
 - Studentas, 13
- getPavarde
 - Studentas, 13
 - Zmogus, 18
- getPazM
 - Studentas, 13
 - Zmogus, 18
- getPazVid
 - Studentas, 14
 - Zmogus, 18
- getReallocationCount
 - ManoVektorius< T >, 10
- getVardas
 - Studentas, 14
 - Zmogus, 18
- istrintiPaskutiniHw
 - Studentas, 14
- iterator
 - ManoVektorius< T >, 8
- iv1
 - Studentas, 14
- iv2
 - Studentas, 14
- iv3
 - Studentas, 14
- iv4
 - Studentas, 14
- main
 - vektoriuMain.cpp, 30
- ManoVektorius
 - ManoVektorius< T >, 8
- ManoVektorius< T >, 7
 - ~ManoVektorius, 8
 - at, 9
 - back, 9
 - begin, 9
 - capacity, 9
 - clear, 9
 - const_iterator, 8
 - empty, 9
 - end, 9, 10
 - erase, 10
 - getReallocationCount, 10
 - iterator, 8
 - ManoVektorius, 8
 - operator=, 10
 - operator[], 10
 - pop_back, 10
 - push_back, 11
 - reallocate, 11
 - size, 11
 - value_type, 8

- mediana
 - Studentas, 14
- operator<<
 - Studentas, 16
 - students.cpp, 24
 - Zmogus.h, 30
- operator>>
 - Studentas, 16
 - students.cpp, 24
- operator=
 - ManoVektorius< T >, 10
 - Studentas, 15
 - Zmogus, 18
- operator[]
 - ManoVektorius< T >, 10
- pavarde
 - Zmogus, 19
- pazymioPridejimas
 - Studentas, 15
- pazymys_mediana
 - Studentas, 15
- pazymys_vidurkis
 - Studentas, 15
- pop_back
 - ManoVektorius< T >, 10
- push_back
 - ManoVektorius< T >, 11
- raide
 - code.h, 21
 - functions.cpp, 22
- reallocate
 - ManoVektorius< T >, 11
- setEgzaminas
 - Studentas, 15
- setHw
 - Studentas, 15
- setPavarde
 - Studentas, 15
 - Zmogus, 18
- setPazM
 - Studentas, 15
- setPazVid
 - Studentas, 16
- setVardas
 - Studentas, 16
 - Zmogus, 18
- size
 - ManoVektorius< T >, 11
- Studentas, 11
 - ~Studentas, 13
 - getEgzaminas, 13
 - getHw, 13
 - getPavarde, 13
 - getPazM, 13
 - getPazVid, 14
 - getVardas, 14
 - istrintiPaskutiniHw, 14
 - iv1, 14
 - iv2, 14
 - iv3, 14
 - iv4, 14
 - mediana, 14
 - operator<<, 16
 - operator>>, 16
 - operator=, 15
 - pazymioPridejimas, 15
 - pazymys_mediana, 15
 - pazymys_vidurkis, 15
 - setEgzaminas, 15
 - setHw, 15
 - setPavarde, 15
 - setPazM, 15
 - setPazVid, 16
 - setVardas, 16
 - Studentas, 13
 - test, 16
 - vidurkis, 16
- students.cpp
 - operator<<, 24
 - operator>>, 24
- test
 - Studentas, 16
- value_type
 - ManoVektorius< T >, 8
- vardas
 - Zmogus, 19
- vektoriai/code.h, 21, 22
- vektoriai/functions.cpp, 22
- vektoriai/my.h, 23
- vektoriai/students.cpp, 24
- vektoriai/students.h, 24, 25
- vektoriai/vector.h, 26
- vektoriai/vektoriuMain.cpp, 29
- vektoriai/Zmogus.h, 30
- vektoriuMain.cpp
 - main, 30
- vidurkis
 - Studentas, 16
- Zmogus, 17
 - ~Zmogus, 17
 - getPavarde, 18
 - getPazM, 18
 - getPazVid, 18
 - getVardas, 18
 - operator=, 18
 - pavarde, 19
 - setPavarde, 18
 - setVardas, 18
 - vardas, 19
 - Zmogus, 17
- Zmogus.h

operator<<, [30](#)